TRADEPILOT · IMPLEMENTATION BRIEF · 2026-04-27

The Action Plan for the Next 4 Weeks

A self-contained, copy-paste-ready instruction sheet. 7 Phase-1 tasks (this week), 4-week observation discipline, decision gate at 2026-05-25, and a 4-gate ML training watchdog to ensure no bad model ever reaches production.

Status: READY TO IMPLEMENT

Phase 1 (this week, 6-9 hrs):
1.1 BULLISH_PREMARKET_SHORT_BLOCK · 1.2 WINNER_RE_ARM · 1.3 TIME_EXIT_TIGHTENING
1.4 Cost Modeling · 1.5 Retire Dead Engines · 1.6 Weekly Stats Tracker · **1.7 ML Training Watchdog**

PHASE 1	PHASE 2	PHASE 3
This week · 7 tasks	4 weeks observation	2026-05-25 gate
WATCHDOG GATES	**FUTURE TASKS GATED**	**SOURCE-OF-TRUTH**
4 (pre/post/promote/drift)	8 unlock conditions	This document

Print and reference · Hand to next Claude session · Update on each phase completion

0. Mission (one paragraph)

Today's research session (Sun-Mon 2026-04-26 → 04-27) produced 3 analyses confirming TradePilot has a real but unproven signal. Two of six engines (v5, v5_classic) pass 95% CI > 0 across 11 trading days. The remaining engines are either too small a sample or actively losing money. The right move is **NOT** another ML rebuild or new engine — it's to (a) ship 3 high-leverage rule fixes this week, (b) retire dead engines, (c) add basic cost modeling, (d) build a weekly stats tracker, then (e) freeze code for 4 weeks while data accumulates. Decision gate on **2026-05-25** decides whether to start the ML rebuild or extend observation.

1. Reference Documents (read for context)

DOC	PURPOSE	WHEN TO CONSULT
`docs/2026-04-27_DEEP_DIVE_60K.md` (+ .pdf)	Why ₹60k wasn't realistic today; tactical gaps identified	If you need to understand the WINNER_RE_ARM rationale
`docs/2026-04-27_SOLUTION_AND_ML_PLAN.md` (+ .pdf)	The 2-track plan (tactical + ML rebuild)	If you need full Track A spec or ML rebuild plan
`docs/2026-04-27_VALIDATION_HONEST.md` (+ .pdf)	Statistical validation against 11-day dataset and 12 published trading books	If you need to defend the timeline / decision gates
`docs/SHORT_ARM_DIAGNOSIS.md`	Earlier 04-24 finding on SHORT-arm scheduling	Background only — superseded

Operating constraints: - ❌ Do NOT modify v5_3 or v4 (we are retiring them) - ❌ Do NOT start any ML training work this month (gated to ≥50 days of data) - ❌ Do NOT add new engine variants (v5_8 is forbidden) - ❌ Do NOT change capital scale (paper book stays at ₹10L) - ✅ DO ship Phase 1 this week, then freeze - ✅ DO follow the existing test pipeline rules (`~/.claude/rules/dp-test-pipeline.md`) - ✅ DO follow agent-safety rules — no sleep-poll, foreground tasks ≤15 min

1A. ML Architecture Truth — Read Before Any ML Work

This section exists because future Claude sessions sometimes assume "v5 doesn't use ML" and propose adding ML to v5. **That assumption is wrong.** The current architecture is already correct — the question is whether to deepen ML's role, and that's gated.

Current chain (already operational)

```
v5 deploy logic
  └─ score_all_stocks()                          ← v5 calls v4
       └─ prototype/v4/composite_scorer.py
            ├─ ML sub-score (LightGBM)           ← contributes ~20-30% to composite
            ├─ Technical sub-score
            ├─ Regime sub-score
            ├─ Volume sub-score
            └─ Other factors
       → composite score per stock
  └─ v5 ranks by composite, applies pool routing, multi-pool re-entry, SHORT logic
```

So "**rules + composite on top of LightGBM" is literally what v5 does today.** v5 has no ML training of its own (`prototype/v5/models/` is empty), but consumes v4's LightGBM via the composite scorer.

The 3 ML integration patterns

PATTERN	DESCRIPTION	STATUS TODAY	WHEN TO ENABLE
1. Sub-score (BLEND)	ML is one of N sub-scores in composite, weight 20-30%	✓ ACTIVE	Always (current baseline)
2. Filter (PRUNE)	ML predicts win probability per signal; drop signals below threshold	✗ Deferred	Only when ML IC ≥ 0.05 (currently 0.024)
3. Sizing (AMPLIFY)	ML confidence scales position size 0.5×-1.5×	✗ Deferred	Only when ML IC ≥ 0.08

The trap to avoid

DO NOT increase ML influence in v5 while ML IC = 0.024 (below the 0.05 tradeable threshold). Adding Pattern 2 or Pattern 3 with a weak ML signal makes results worse, not better:

- Pattern 2 with bad ML: drops real winning signals at roughly the same rate as fake ones (near-random pruning) → fewer trades, same hit rate, lower P&L
- Pattern 3 with bad ML: amplifies noise, magnifies wrong-side bets → bigger losses on misranked confident-but-wrong calls
- Increasing the ML sub-score weight in the composite from 25% → 50%: dilutes the rule-based alpha that's actually driving today's positive returns

Lopez de Prado calls this "performance theater" in *Advances in Financial Machine Learning* (ch. 11): adding architectural complexity to systems whose underlying signal hasn't been validated. **The right discipline: fix the ML signal first (Track B rebuild), THEN integrate it more aggressively.**

Why v5 currently makes money "despite" the ML

ENGINE	VALIDATION P&L	DRIVER
v4 (pure ML deploy)	-₹2,981/day mean	ML alone — IC=0.024, barely above noise
v5 (composite + rules)	+₹16,921/day mean	Rule-based machinery (multi-pool, SWING re-entry, regime gates). ML is a passenger.

Today, v5's profitability is *despite* the ML, not because of it. This will reverse once ML IC clears 0.05 — but not before. Until then, **don't give a passenger the steering wheel.**

Future tasks (gated, do NOT start in Phase 1)

These patterns are **planned post-validation** and tracked here so they don't get forgotten:

- **TASK FUTURE.1** — Pattern 2 (Filter Mode): Add ML probability filter to `scripts/v5-paper-trade.py:348` deploy loop. Threshold tunable, default 0.55. Drop signals below threshold. **Unlock condition: ML IC ≥ 0.05 confirmed for 30 consecutive trading days.**
- **TASK FUTURE.2** — Pattern 3 (Confidence-Sized Positions): Multiply position size by `clip(ml_confidence, 0.5, 1.5)`. **Unlock condition: ML IC ≥ 0.08 confirmed for 30 consecutive trading days.**
- **TASK FUTURE.3** — Composite Weight Increase: Raise ML sub-score weight in `composite_scorer.py` from 25% to 40%. **Unlock condition: ML IC ≥ 0.10 AND watchdog has rejected zero promotions in last 60 days.**

These tasks are explicitly out-of-scope for Phase 1 and Phase 2. Phase 3 decision gate (2026-05-25) determines if/when they unlock.

---

2. Phase 1 — This Week (6-9 hours total)

Seven tasks. Do them in this order.

TASK 1.1 — Implement BULLISH_PREMARKET_SHORT_BLOCK

Estimated effort: 45 min **File:** `scripts/v5-paper-trade.py`

What: When premarket bias is BULLISH and gap up > 0.5%, suppress all SELL/SHORT signals for the first 60 minutes of trading.

Why: On 04-27 the engine took 36 SHORTs in a gap-up bullish session and 18 hit STOPLOSS, costing ~₹2,100 per engine. Premarket told us BULLISH explicitly — engine ignored it.

Where to insert: Just before the deploy-loop sort at line 348 of `scripts/v5-paper-trade.py` :

```
# === BULLISH_PREMARKET_SHORT_BLOCK (added 2026-04-28) ===
def _short_block_active(state):
    pm = state.get("premarket", {})
    bias_bullish = pm.get("bias", "").upper() == "BULLISH"
    gap = pm.get("gap_prediction", {})
    gap_up = (gap.get("direction") == "UP" and
              float(gap.get("magnitude_pct", 0) or 0) > 0.5)
    # First 60 min after open (09:15 IST + 60 min = 10:15)
    from datetime import datetime
    minutes_since_open = (datetime.now().hour - 9) * 60 + datetime.now().minute - 15
    in_window = 0 <= minutes_since_open < 60
    return bias_bullish and gap_up and in_window

allowed_dirs = ("BUY",) if _short_block_active(state) else ("BUY", "SELL")
if _short_block_active(state):
    log(f"  [SHORT_BLOCK] Bullish premarket + gap-up — suppressing SELL signals for first 60 min")

for sig in sorted([s for s in signals if s["direction"] in allowed_dirs],
                  key=lambda s: -float(s.get("score", 0))):
    # ... existing loop body unchanged ...
```

Acceptance criteria: - [ ] Code compiles, engine starts without error - [ ] Log line `[SHORT_BLOCK]` appears on next bullish-gap-up morning - [ ] When block is inactive, all SELL signals proceed as before (no behavior change on flat/bear days) - [ ] Make `0.5` (gap threshold) and `60` (window minutes) read from `os.environ.get("SHORT_BLOCK_GAP_PCT", "0.5")` and `SHORT_BLOCK_WINDOW_MIN` so they're tunable without code changes

---

TASK 1.2 — Implement WINNER_RE_ARM

Estimated effort: 90 min **File:** `scripts/v5-paper-trade.py`

What: When a position exits with `reason == "TARGET"` , mark the symbol as "re-armable" with counter=3. On next rescore, if the same symbol generates a fresh signal in the same direction, allow re-deployment.

Why: This is the entire 04-22 ₹61k mechanic. MOTHERSON re-entered 8 times that day, IREDA 7 times, ABB 6 times. 8 stocks generated ~₹35,600 of the ₹61k by re-arming on TARGET hits. On 04-27, SAIL/SUZLON/JSWENERGY hit TARGET as single-entry trades — no re-arm logic exists.

Implementation steps:

1. Add helper function near `close_position()`:

```
def mark_rearmable(state, symbol, direction, max_rearms=3):
    """When a position exits with TARGET, allow up to N re-entries today."""
    rearm = state.setdefault("rearmable", {})
    if symbol not in rearm:
        rearm[symbol] = {
            "direction": direction,
            "remaining": max_rearms,
            "expires_at_minute": (15 - 9) * 60  # 15:00 IST cutoff in minutes-since-open
        }
    return rearm[symbol]

def consume_rearm(state, symbol, direction):
    """Returns True if a re-arm slot is available and consumes it."""
    rearm = state.get("rearmable", {}).get(symbol)
    if not rearm or rearm["direction"] != direction or rearm["remaining"] <= 0:
        return False
    rearm["remaining"] -= 1
    return True
```

2. In `close_position()`, after computing `pnl`, before logging:

```
if reason == "TARGET":
    direction = "BUY" if not is_short else "SELL"
    mark_rearmable(state, sym, direction)
    log(f"  {sym}: TARGET hit — re-armable for {direction} (3 slots)")
```

3. In `deploy_signals()`, modify the `if sym in held` early-skip at line 351. Currently it skips if symbol is already held. New logic: also allow if re-arm available.

```
# Existing: if sym in held or pool_name not in state["pools"] or pool_name == "NONE": continue
# Modified:
if pool_name not in state["pools"] or pool_name == "NONE":
    continue
already_held = sym in held
rearm_ok = consume_rearm(state, sym, sig["direction"]) if already_held else False
if already_held and not rearm_ok:
    continue  # skip — already held and no re-arm available
if rearm_ok:
    log(f"  [RE-ARM] {sym}: deploying re-entry on {sig['direction']}")
```

Acceptance criteria: - [ ] When a stock exits with TARGET, log shows "re-armable for BUY/SELL (3 slots)" - [ ] On next rescore, if same stock signals same direction, log shows "[RE-ARM] {sym}: deploying re-entry" - [ ] Maximum 3 re-arms per stock per day (4th attempt should silently skip) - [ ] Re-arms ONLY trigger on TARGET exit, never on STOPLOSS or TIME_EXIT (verify by inspecting `close_position` callsites) - [ ] Unit test: simulate TARGET exit + same-direction signal → expect deploy - [ ] Unit test: simulate STOPLOSS exit + same-direction signal → expect skip (no re-arm) - [ ] Test the existing `is_reentry_blocked` (2-SL same-day block from learning 2026-04-17_003) still works — it should NOT be bypassed by re-arm

---

TASK 1.3 – Implement TIME_EXIT_TIGHTENING

Estimated effort: 30 min **File:** `scripts/v5-paper-trade.py`

What: At 1:30 PM IST, force-exit any position with `|unrealized_pnl_pct| < 0.3%` (i.e., flat positions) to free the slot.

Why: On 04-27, 18 of 58 trades exited as TIME_EXIT — flat positions held all day with avg P&L ~₹0-30. They locked capital that fresh signals could have used.

Where: In `manage_positions()` loop. Add a time check near the start of position iteration:

```
# === TIME_EXIT_TIGHTENING (added 2026-04-28) ===
from datetime import datetime
now = datetime.now()
post_lunch_window = (now.hour, now.minute) >= (13, 30) and (now.hour, now.minute) < (14, 0)

for pos in pool["positions"][:]:  # copy for safe mutation
    sym = pos["symbol"]
    is_short = pos.get("position_type") == "SHORT"
    current_price = get_live_price(sym)  # use existing helper
    pnl_pct = ((current_price - pos["entry_price"]) / pos["entry_price"] * 100
               * (-1 if is_short else 1))
    if post_lunch_window and abs(pnl_pct) < 0.3:
        log(f"  {sym}: FLAT_FORCE_EXIT @ {current_price:.2f} (pnl_pct={pnl_pct:.2f}%)")
        close_position(state, pm, rm, pool_name, pos, current_price, "FLAT_FORCE_EXIT")
        continue
    # ... existing SL/target/trailing logic ...
```

Acceptance criteria: - [ ] At 13:30-14:00 IST, any position with `abs(pnl_pct) < 0.3%` exits with reason `FLAT_FORCE_EXIT` - [ ] Reason `FLAT_FORCE_EXIT` is added to the report.md exit-type breakdown - [ ] Threshold `0.3` is tunable via `os.environ.get("FLAT_EXIT_THRESHOLD_PCT", "0.3")` - [ ] Window `13:30-14:00` tunable via `FLAT_EXIT_WINDOW_START` / `FLAT_EXIT_WINDOW_END` - [ ] Existing time-of-day-experiment data collection (per memory `project_tradepilot_time_experiment.md`) is NOT broken by this change

---

TASK 1.4 – Add Cost Modeling

Estimated effort: 30 min **File:** `scripts/v5-paper-trade.py` + `docs/paper-trades/{engine}/{date}_report.md` template

What: Subtract realistic Indian retail brokerage + STT + slippage from every closed trade's P&L. Show both gross and net P&L in the daily report.

Why: The honest validation showed paper P&L is misleading without cost modeling. Live trades will lose 10-12 bps round-trip per trade. With ~50 trades/day = ₹600/day cost drag. Reporting net P&L makes paper match live behavior.

Implementation:

```
# Round-trip cost in basis points (Indian retail intraday)
COST_BPS_ROUND_TRIP = float(os.environ.get("COST_BPS_ROUND_TRIP", "12"))

def cost_for_trade(qty, entry_price, exit_price):
    """Realistic cost: brokerage + STT + slippage in INR."""
    notional_avg = qty * (entry_price + exit_price) / 2
    return notional_avg * (COST_BPS_ROUND_TRIP / 10000)
```

In `close_position()`, after computing gross `pnl`:

```
cost = cost_for_trade(pos["qty"], pos["entry_price"], exit_price)
pnl_net = pnl - cost
pos["cost"] = cost
pos["pnl_gross"] = pnl
pos["pnl_net"] = pnl_net
```

In the daily report generator (search for `Net P&L` markdown line): - Keep `Net P&L` as gross for backwards compatibility with prior reports - Add a new line `**P&L after costs**: Rs {sum_pnl_net} (gross Rs {sum_pnl_gross}, costs Rs {sum_cost})`

Acceptance criteria: - [ ] Each closed trade has `pnl_gross`, `pnl_net`, `cost` fields in the JSON snapshot - [ ] Daily report shows both gross and net P&L - [ ] At default 12 bps, today's v5 report (₹737 gross) should show ~₹100-200 in costs and net ~₹500-600 - [ ] Threshold `COST_BPS_ROUND_TRIP` is configurable via env var - [ ] Verify by spot-checking 3 trades manually: cost ≈ 0.0012 × notional_avg

---

TASK 1.5 — Retire Dead Engines

Estimated effort: 15 min

What: Stop running v5_3 (10 days, 1 win, mean ₹-19) and v4 ML (2 days, mean ₹-2,981). Keep their code in the repo for reference but disable their schedulers.

Why: Both engines are statistically dead. v5_3 is ML-driven and uses an empty model dir (`prototype/v5/models/` is empty). v4 has IC=0.024 — confirmed not tradeable. Running them is pure overhead.

Steps: 1. Find the launchd / cron / systemd configs that schedule each engine. Likely in `~/Library/LaunchAgents/` or `scripts/` (look for `*v5_3*` and `*v4*` schedulers). 2. Disable (don't delete) the schedulers: `bash launchctl unload ~/Library/LaunchAgents/com.tradepilot.v5_3.plist launchctl unload ~/Library/LaunchAgents/com.tradepilot.v4.plist` 3. Add a one-line note to `docs/RETIRED_ENGINES.md` (create if missing): `- v5_3 retired 2026-04-28: 1/10 win days, mean ₹-19/day, 95% CI contains zero - v4 (ML composite) retired 2026-04-28: 0/2 win days, IC=0.024 below tradeable threshold`

Acceptance criteria: - [ ] No new logs appear in `logs/v5_3-*.log` or `logs/v4-*.log` after retirement date - [ ] `docs/paper-trades/v5_3/` and `docs/paper-trades/v4/` stop receiving new daily reports - [ ] Retirement is reversible (just `launchctl load` again) - [ ] Document retirement in `docs/RETIRED_ENGINES.md`

TASK 1.6 — Build the Weekly Stats Tracker

Estimated effort: 1 hour **File:** `scripts/weekly-stats-tracker.py` (new)

What: A 50-line Python script that runs every Monday morning, reads all `docs/paper-trades/{engine}/*_report.md` files since 2026-04-10, and prints aggregate statistics for each surviving engine.

Why: Need ONE number to glance at every Monday. No dashboard, no UI — just a printout. Watching the stats catch up with the ambition is the whole point of the 4-week observation phase.

Spec:

```
#!/usr/bin/env python3
"""Weekly TradePilot stats tracker.
Run every Monday morning. Outputs cumulative P&L, deflated Sharpe, and 95% CI per engine.
Usage: python3 scripts/weekly-stats-tracker.py
"""
import re
import numpy as np
from pathlib import Path
from scipy import stats as scipy_stats

PT = Path(__file__).parent.parent / "docs/paper-trades"
ENGINES = ['v5', 'v5_6', 'v5_7', 'v5_classic']  # surviving engines only
PNL_RE = re.compile(r"Net P&L\*\*\s*\|\s*\*\*Rs\s*([-\d,]+)")
WIN_RE = re.compile(r"Win Rate\s*\|\s*([\d]+)%")
CAPITAL = 1_000_000  # ₹10L paper book

def load_engine(engine):
    rows = []
    for f in sorted((PT / engine).glob("2026-*_report.md")):
        m = PNL_RE.search(f.read_text())
        w = WIN_RE.search(f.read_text())
        if m:
            rows.append({
                "date": f.stem.replace("_report", ""),
                "pnl": int(m.group(1).replace(",", "")),
                "win_rate": int(w.group(1)) if w else None
            })
    return rows

def deflated_sharpe(pnls, n_trials=4):
    """Lopez de Prado's Deflated Sharpe Ratio (simplified).
    Adjusts for selection bias (n_trials engines) and sample size."""
    pnls = np.array(pnls)
    if len(pnls) < 2:
        return None
    daily_ret = pnls / CAPITAL
    raw_sharpe = daily_ret.mean() / daily_ret.std() * np.sqrt(252) if daily_ret.std() > 0 else 0
    # Selection bias haircut: ~25% per engine beyond first
    selection_haircut = 1 - 0.05 * (n_trials - 1)
    # Sample size haircut: shrinks as n grows toward 252
    sample_haircut = min(1.0, len(pnls) / 252) ** 0.5
    return raw_sharpe * selection_haircut * sample_haircut

def main():
    print(f"\n{'='*80}\nTradePilot Weekly Stats Tracker\n{'='*80}\n")
    print(f"{'Engine':<12} {'Days':<5} {'Total P&L':<14} {'Mean/day':<12} "
          f"{'95% CI':<28} {'Raw Sharpe':<11} {'Defl. Sharpe':<12} {'Win days':<10}")
    print('-' * 110)
    for eng in ENGINES:
        rows = load_engine(eng)
        if not rows:
            continue
        pnls = np.array([r["pnl"] for r in rows])
        n = len(pnls)
        mean = pnls.mean()
        std = pnls.std()
        if n >= 2:
            t_crit = scipy_stats.t.ppf(0.975, n - 1)
            ci_low = mean - t_crit * std / np.sqrt(n)
            ci_high = mean + t_crit * std / np.sqrt(n)
            ci_str = f"[₹{ci_low:>9,.0f}, ₹{ci_high:>9,.0f}]"
        else:
            ci_str = "n<2"
```

```
        sharpe = (pnls / CAPITAL).mean() / (pnls / CAPITAL).std() * np.sqrt(252) if std > 0 else 0
        defl = deflated_sharpe(pnls, n_trials=len(ENGINES))
        win_days = (pnls > 0).sum()
        sig = " ✓" if (n >= 2 and ci_low > 0) else ""
        print(f"{eng:<12} {n:<5} ₹{pnls.sum():>11,} ₹{mean:>9,.0f} {ci_str:<28} "
              f"{sharpe:>9.2f} {(defl or 0):>10.2f} {win_days}/{n}{sig}")
    print(f"\n{'='*80}\nDecision-gate criteria for 2026-05-25:")
    print(f"  Engine passes if: 95% CI > 0 AND days >= 30 AND Defl. Sharpe >= 2.0")
    print(f"{'='*80}\n")

if __name__ == "__main__":
    main()
```

Acceptance criteria: - [ ] Script runs in <5 seconds - [ ] Output is a single ASCII table (no JSON, no dashboard) - [ ] Handles missing reports gracefully (no crash on partial weeks) - [ ] Decision-gate criteria printed at the bottom every run - [ ] Add a `0 9 * * MON` cron entry OR a launchd plist that runs it Monday 9 AM and pipes to `logs/weekly-stats-{date}.txt` - [ ] Manually test on current data — should show v5 (11 days), v5_6 (7), v5_7 (7), v5_classic (6)

TASK 1.7 – Build the ML Training Watchdog

Estimated effort: 2-3 hours **Files:** `scripts/ml-training-watchdog.py` (new), `scripts/ml-approve.sh` (new), modify `scripts/retrain-ml.sh`

What: A 4-gate quality system that prevents bad/corrupted ML training data from producing a deployed model. **NO new model auto-promotes to production.** Every retrain output goes to `prototype/v4/models/staging/` and requires explicit human approval before it can be loaded by the engines.

Why: Per §1A architecture truth, v5's profitability depends on v4's ML score being *reasonable* (not actively bad). A silent training failure — data corruption, schema change, bug in feature pipeline, look-ahead leakage — could degrade v4's ML score to *below* noise (negative IC), which would propagate into v5's composite scores, which would degrade v5's profitability invisibly. The watchdog catches these failures before the bad model goes live.

The user's directive (from 2026-04-27 conversation): "*no misinformation or wrong training should go to the ML until we confirm it*" — this means human-in-the-loop approval is mandatory, no auto-promote.

Gate 1 – Pre-training data validation

Run BEFORE training starts. If any check fails, ABORT — don't even start training.

```
# scripts/ml-training-watchdog.py — pre_training_checks()

CRITICAL_FEATURES = ['india_vix', 'nifty_change_pct', 'atr_norm', 'gap_pct',
                     'prev_day_range_pct', 'rsi_14', 'volume_ratio_20d']

def pre_training_checks(training_df):
    """Returns (passed: bool, reasons: list[str])."""
    checks = []

    # 1. No NaN in critical features
    nan_counts = training_df[CRITICAL_FEATURES].isna().sum()
    nan_cols = nan_counts[nan_counts > 0]
    checks.append(("no_nan_in_critical", len(nan_cols) == 0,
                   f"NaN in: {nan_cols.to_dict()}"))

    # 2. No future-dated rows (catches accidental data leakage)
    today = pd.Timestamp.today().normalize()
    future = (pd.to_datetime(training_df['date']) > today).sum()
    checks.append(("no_future_dates", future == 0,
                   f"Found {future} rows with date > today"))

    # 3. Target distribution sanity (no >30% intraday returns — likely bad data)
    target_extreme = (training_df['target'].abs() > 0.30).sum()
    checks.append(("no_extreme_returns", target_extreme < 10,
                   f"Found {target_extreme} rows with |return| > 30% (likely bad ticks)"))

    # 4. Sample size sanity (50K is typical; <30K = something wrong)
    checks.append(("sample_size_adequate", len(training_df) > 30000,
                   f"Only {len(training_df)} rows — too few"))

    # 5. Date range coverage (training should span 12+ months for stationarity)
    date_span_days = (training_df['date'].max() - training_df['date'].min()).days
    checks.append(("date_span_adequate", date_span_days > 365,
                   f"Only {date_span_days} days of data — too short"))

    # 6. Feature distribution drift vs last training run
    prev_stats = load_previous_stats()  # JSON in models/training_stats.json
    if prev_stats:
        for f in CRITICAL_FEATURES:
            if f not in training_df.columns:
                continue
            current_mean = float(training_df[f].mean())
            previous_mean = prev_stats.get(f, {}).get('mean')
            if previous_mean and abs(previous_mean) > 1e-6:
                drift_pct = abs(current_mean - previous_mean) / abs(previous_mean) * 100
                checks.append((f"drift_{f}", drift_pct < 30,
                               f"{f} mean drifted {drift_pct:.0f}% (current={current_mean:.4f}, prev=
{previous_mean:.4f})"))

    # 7. No look-ahead in target (target_date should always be > feature_date)
    if 'feature_date' in training_df.columns and 'target_date' in training_df.columns:
        leakage = (training_df['target_date'] <= training_df['feature_date']).sum()
        checks.append(("no_lookahead_leakage", leakage == 0,
                       f"Found {leakage} rows with target_date <= feature_date"))

    passed = all(c[1] for c in checks)
    failures = [(name, msg) for name, ok, msg in checks if not ok]
    return passed, failures
```

Gate 2 – Post-training model validation

Run AFTER training completes, BEFORE writing the new model anywhere. If any check fails, the new model is DISCARDED — not saved to staging, not saved anywhere.

```
def post_training_checks(new_model, validation_df, current_production_model):
    """Compare new model against current production. Returns (passed, failures)."""
    checks = []

    # Compute IC on the same out-of-sample window for both models
    new_ic = compute_information_coefficient(new_model, validation_df)
    baseline_ic = compute_information_coefficient(current_production_model, validation_df)

    # 1. New IC must be > 0 (basic sanity – random model has IC ~ 0)
    checks.append(("ic_positive", new_ic > 0,
                   f"New IC = {new_ic:.4f} (must be > 0)"))

    # 2. New IC must not be DRAMATICALLY worse than baseline (allow small regression for natural
variance)
    ic_drop_pct = ((baseline_ic - new_ic) / baseline_ic * 100) if baseline_ic > 0 else 0
    checks.append(("ic_not_degraded", ic_drop_pct < 30,
                   f"IC dropped {ic_drop_pct:.0f}% (new={new_ic:.4f}, baseline={baseline_ic:.4f})"))

    # 3. Top features should not radically shift (3 of top-5 should overlap with previous)
    new_top5 = top_features(new_model, n=5)
    baseline_top5 = top_features(current_production_model, n=5)
    overlap = len(set(new_top5) & set(baseline_top5))
    checks.append(("feature_stability", overlap >= 3,
                   f"Only {overlap}/5 top features overlap with baseline. New: {new_top5}"))

    # 4. Hit rate on holdout shouldn't crash by >10pp
    new_hit = hit_rate(new_model, validation_df)
    baseline_hit = hit_rate(current_production_model, validation_df)
    hit_drop = baseline_hit - new_hit
    checks.append(("hit_rate_stable", hit_drop < 0.10,
                   f"Hit rate dropped {hit_drop:.1%} (new={new_hit:.1%}, baseline=
{baseline_hit:.1%})"))

    # 5. Walk-forward IC positive folds: at least 50% (currently 53%)
    wf_pos_pct = walk_forward_positive_pct(new_model, validation_df)
    checks.append(("walk_forward_stable", wf_pos_pct >= 0.50,
                   f"Only {wf_pos_pct:.0%} of walk-forward folds had positive IC"))

    passed = all(c[1] for c in checks)
    failures = [(name, msg) for name, ok, msg in checks if not ok]
    return passed, failures
```

Gate 3 – Promotion (manual approval, NEVER automatic)

Models that pass Gates 1+2 are saved to `prototype/v4/models/staging/lgbm_intraday_YYYY-MM-DD.txt`. **They are NOT loaded by any engine.** A human runs:

```
# scripts/ml-approve.sh
#!/bin/bash
# Promotes a staged model to production after manual review.
# Usage: ./scripts/ml-approve.sh <staging_filename>

set -e
STAGING_DIR="prototype/v4/models/staging"
PRODUCTION_PATH="prototype/v4/models/lgbm_intraday.txt"
ARCHIVE_DIR="prototype/v4/models/archive/$(date +%Y-%m-%d)"

if [ -z "$1" ]; then
    echo "Usage: $0 <staging_filename>"
    echo ""
    echo "Available staged models:"
    ls -la "$STAGING_DIR" 2>/dev/null
    exit 1
fi

STAGED_FILE="$STAGING_DIR/$1"
[ -f "$STAGED_FILE" ] || { echo "Error: $STAGED_FILE not found"; exit 1; }

# Show diff in metrics between staged and production
echo "=== Promotion Review ==="
echo "Production: $PRODUCTION_PATH"
echo "Staged:     $STAGED_FILE"
echo ""
python3 scripts/ml-training-watchdog.py --compare "$STAGED_FILE" "$PRODUCTION_PATH"
echo ""

# Hard-coded confirmation (no auto-yes)
read -p "Type 'PROMOTE' to confirm (any other input cancels): " response
if [ "$response" != "PROMOTE" ]; then
    echo "Cancelled. No changes made."
    exit 1
fi

# Archive current production
mkdir -p "$ARCHIVE_DIR"
cp "$PRODUCTION_PATH" "$ARCHIVE_DIR/lgbm_intraday.txt"
cp "prototype/v4/models/lgbm_meta.json" "$ARCHIVE_DIR/" 2>/dev/null || true
echo "Archived old model to: $ARCHIVE_DIR/"

# Promote staged model
cp "$STAGED_FILE" "$PRODUCTION_PATH"
echo "Promoted: $1 → $PRODUCTION_PATH"
echo ""
echo "Next: restart engines to load new model:"
echo "  launchctl kickstart -k gui/$(id -u)/com.tradepilot.v5"
echo ""
echo "Watchdog will compute live IC daily; check logs/ml-drift-*.log for first-week stability."
```

Gate 4 — Live drift monitor (runs daily during Phase 2 and beyond)

After every trading day, compute the actual IC of the deployed model on that day's signals (predicted vs realized outcomes). Track rolling 5-day IC. If it goes negative, alert.

```
def daily_drift_check(date):
    """Compute today's actual IC on live signals. Alert if rolling IC degrades."""
    signals = load_signals_for_date(date)         # ml_score per signal
    outcomes = load_outcomes_for_date(date)       # actual TARGET hit (1/0)

    if len(signals) < 10:
        log("Insufficient signals for IC computation today")
        return

    today_ic = pearsonr(signals['ml_score'], outcomes['won'])[0]

    # Append to rolling log
    log_path = LOG_DIR / "ml-drift-rolling.json"
    rolling = json.load(log_path.open()) if log_path.exists() else []
    rolling.append({'date': str(date), 'ic': today_ic, 'n_signals': len(signals)})
    rolling = rolling[-30:]  # keep last 30 days
    json.dump(rolling, log_path.open('w'), indent=2)

    # Compute rolling 5-day mean IC
    last5 = rolling[-5:]
    rolling_mean = sum(r['ic'] for r in last5) / len(last5)

    log(f"Today IC: {today_ic:.4f} | 5-day rolling: {rolling_mean:.4f}")

    if rolling_mean < 0.0:
        alert(f"ML DRIFT ALERT: rolling 5-day IC = {rolling_mean:.4f} (NEGATIVE — investigate)")
        # Optional: auto-disable ML sub-score in composite by writing a flag file
        # (prototype/v4/models/.ml_disabled)
```

Wiring it all together

Modify `scripts/retrain-ml.sh` (existing) to call the watchdog at the right phases:

```
#!/bin/bash
# scripts/retrain-ml.sh — updated to use watchdog

set -e

echo "=== Step 1: Pre-training data validation ==="
python3 scripts/ml-training-watchdog.py --pre || {
    echo "PRE-CHECK FAILED. Training aborted. Check logs/ml-watchdog.log for reasons."
    exit 1
}

echo "=== Step 2: Training new model ==="
python3 -m prototype.v4.ml_engine --train --output-staging
# ml_engine.py modified to write to staging/ directory, not production directly

echo "=== Step 3: Post-training model validation ==="
python3 scripts/ml-training-watchdog.py --post || {
    echo "POST-CHECK FAILED. New model rejected. Production model unchanged."
    rm prototype/v4/models/staging/lgbm_intraday_$(date +%Y-%m-%d).txt
    exit 1
}

echo ""
echo "=== Both gates passed ==="
echo "Staged model: prototype/v4/models/staging/lgbm_intraday_$(date +%Y-%m-%d).txt"
echo ""
echo "Production model UNCHANGED. To promote:"
echo "  ./scripts/ml-approve.sh lgbm_intraday_$(date +%Y-%m-%d).txt"
```

Schedule

- **Pre-training + post-training gates**: run on every retrain trigger (existing schedule, monthly)
- **Live drift monitor**: launchd plist runs daily at 16:00 IST after market close, after position closure logs are written

```
<!-- ~/Library/LaunchAgents/com.tradepilot.ml-drift.plist -->
<plist>
<dict>
  <key>Label</key><string>com.tradepilot.ml-drift</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/soumyaswain/anaconda3/bin/python3</string>
    <string>/Users/soumyaswain/Documents/tinker/projects/tradepilot/scripts/ml-training-
watchdog.py</string>
    <string>--drift-check</string>
  </array>
  <key>StartCalendarInterval</key>
  <dict>
    <key>Hour</key><integer>16</integer>
    <key>Minute</key><integer>0</integer>
    <key>Weekday</key><integer>1-5</integer>
  </dict>
  <key>StandardOutPath</key>
  <string>/Users/soumyaswain/Library/Logs/tradepilot-ml-drift.log</string>
  <key>StandardErrorPath</key>
  <string>/Users/soumyaswain/Library/Logs/tradepilot-ml-drift.err</string>
</dict>
</plist>
```

(Note: stdout/stderr paths in `~/Library/Logs/`, NOT `~/Documents/` — per `project_omnipilot_launchagent.md` memory: macOS TCC blocks Documents path, exit code 78, silent failure.)

Acceptance criteria

- [ ] `prototype/v4/models/staging/` directory created
- [ ] `scripts/ml-training-watchdog.py` runs both pre and post gates and produces clear pass/fail output
- [ ] `scripts/ml-approve.sh` requires explicit "PROMOTE" string confirmation (no `-y` flag, no env override)
- [ ] `scripts/retrain-ml.sh` modified to call watchdog at correct phases
- [ ] If pre-check fails: training is aborted (no model produced)
- [ ] If post-check fails: no model saved (not even to staging)
- [ ] If both pass: model goes to staging only — production file `prototype/v4/models/lgbm_intraday.txt` unchanged
- [ ] Daily drift check launchd plist created and loaded
- [ ] Rolling IC log at `logs/ml-drift-rolling.json` updated daily
- [ ] If rolling 5-day IC < 0.0: alert logged AND optionally write flag file `.ml_disabled` to disable ML sub-score (composite scorer reads this flag)
- [ ] **Test 1**: corrupt a feature in training data (set 50% to NaN) → verify Gate 1 blocks the run
- [ ] **Test 2**: train with shuffled labels → verify Gate 2 rejects the resulting model (IC will be ~0)
- [ ] **Test 3**: promote a staged model with `ml-approve.sh` → verify production file changes only after typing "PROMOTE"
- [ ] **Test 4**: simulate 5 consecutive days of fake-negative IC → verify drift alert fires
- [ ] Quick-status command works: `python3 scripts/ml-training-watchdog.py --status` shows current production model + last 5 staged models + rolling IC trend
- [ ] Add to `MEMORY.md` quick-reference: "ml-watchdog --status" command

Why this is critical for the next 4 weeks

During Phase 2 (observation window), the monthly retrain WILL run automatically. Without the watchdog, a corrupted retrain could silently overwrite the production model — invalidating the entire 4-week observation experiment. The watchdog ensures the production model **cannot change** during Phase 2 unless a human explicitly promotes it. This protects the validity of the decision-gate analysis at 2026-05-25.

Phase 1 Sign-Off Checklist

Before moving to Phase 2, verify ALL of these:

- [ ] Tasks 1.1, 1.2, 1.3, 1.4 implemented in `scripts/v5-paper-trade.py`
- [ ] Code committed with message: `feat(engines): Track A tactical fixes + cost modeling (DP-TRADE-2026-04-28)`
- [ ] All four v5-family engines (v5, v5_6, v5_7, v5_classic) start without error
- [ ] One paper-trading day completes successfully under the new logic (Tuesday 2026-04-28)
- [ ] v5_3 and v4 schedulers disabled (Task 1.5)
- [ ] Weekly tracker script runs and produces correct output (Task 1.6)
- [ ] **ML training watchdog (Task 1.7) implemented and tested with all 4 acceptance tests**
- [ ] **Production model `lgbm_intraday.txt` is now write-protected by the watchdog flow**
- [ ] `prototype/v4/models/staging/` **directory exists**
- [ ] Cron / launchd entry added for Monday tracker run AND daily drift check (16:00 IST)
- [ ] `docs/RETIRED_ENGINES.md` created with v5_3 and v4 entries
- [ ] Update `MEMORY.md` with two new entries:
- "Track A shipped 2026-04-28 — observation phase begins"
- "ML watchdog active — no auto-promote, manual approval required"
- [ ] Run `dp learn "TRACK_A_SHIPPED: ..."` and `dp learn "ML_WATCHDOG_ACTIVE: ..."` to record milestones in DevPilot DB

---

3. Phase 2 — 4-Week Observation (2026-04-29 → 2026-05-25)

The hard rule: NO ENGINE CODE CHANGES during this window.

This is a discipline test, not a coding task. Bug fixes that don't affect trading behavior (logging, doc improvements, infra) are allowed. Anything that touches signal generation, deploy logic, position sizing, exits, risk gates is forbidden.

Daily routine (≤5 min)

- Engines run on auto-schedule (already in place).
- Daily reports auto-generate (already in place).
- **Do not touch the code.**

Weekly routine (every Monday, ≤15 min)

1. Run `python3 scripts/weekly-stats-tracker.py`
2. Glance at output. Note anything surprising.
3. Optional 1-2 line journal entry in `docs/observation_journal.md`: `## 2026-05-04 — Week 1 of observation - v5: 14 days now, mean ₹14,2k, 95% CI [+₹3.8k, +₹24.6k] — still significant - First drawdown`

```
observed Thursday (-₹3,200), recovered Friday - v5_6 mean down slightly — small sample noise
```

What you're watching for

SIGNAL	MEANING
First 5%+ peak-to-trough drawdown	Required validation event — proves system can recover
First BEAR-regime day	Out-of-distribution test — could happen any week
Sharpe regression toward 1.5-3.5	Healthy — confirms the 13-20 was small-sample inflation
95% CI continues to exclude zero (v5/v5_classic)	Signal is real
95% CI starts to exclude zero (v5_6/v5_7)	They cross the line into "validated"
Sudden Sharpe drop below 0.5	Concerning — investigate (but DO NOT change code mid-window)

What you do NOT do

- ❌ Tune any parameter (gap threshold, re-arm count, cost bps)
- ❌ Add new features or engines
- ❌ Restart ML training
- ❌ Change capital scale
- ❌ Panic-disable an engine on a single bad day
- ❌ Cherry-pick best window for reporting

---

4. Phase 3 – Decision Gate (2026-05-25)

On 2026-05-25 (Monday morning), run the weekly tracker. Apply this decision matrix:

The 4 gate criteria

#	CRITERION	THRESHOLD
1	v5 95% CI > 0	Excludes zero AND lower bound > ₹3,000
2	v5_6 OR v5_7 95% CI > 0	At least one of them clears
3	Drawdown observed	At least one 5%+ peak-to-trough event recorded AND recovered within 5 days
4	Deflated Sharpe ≥ 2.0	For v5 specifically (largest sample)

Decision matrix

# OF CRITERIA MET	ACTION
4 of 4	Start ML rebuild Track B (per `SOLUTION_AND_ML_PLAN.md` §2). Begin small-scale live trading at ₹50k (5% of paper capital).
3 of 4	Extend observation by 30 days. No code changes. Re-evaluate 2026-06-25.
2 of 4	Investigate root cause of underperformance. Possibly revert Track A changes if they correlate with degradation.
0-1 of 4	Pause TradePilot indefinitely. Reallocate engineering effort to BizBot / Dhanvantari / OmniPilot. Revisit in Q4 with fresh perspective.

Honesty check at the gate

Re-read `docs/2026-04-27_VALIDATION_HONEST.md` §6 (book comparisons). Compare current state to the realistic benchmarks: - Sharpe should regress toward 1.5-3.5 - Win-day rate should land 55-75% - Annualized projection should fall to 30-150% (still excellent) - Any number outside these ranges = small-sample noise, not signal

---

5. Out-of-Scope – Things This Brief Does NOT Include

These are deliberately NOT in scope, with explicit **unlock conditions** so future Claude sessions know exactly when each becomes valid.

Hard-gated future work (refer here when user asks)

TASK	DESCRIPTION	UNLOCK CONDITION	REFERENCE DOC
TASK FUTURE.1	Pattern 2 — ML filter mode in v5 deploy loop	ML IC ≥ 0.05 confirmed for 30 consecutive trading days	§1A + SOLUTION doc §2.5
TASK FUTURE.2	Pattern 3 — Confidence-sized positions	ML IC ≥ 0.08 confirmed for 30 consecutive trading days	§1A + SOLUTION doc §2.5
TASK FUTURE.3	Composite weight increase (ML 25% → 40%)	ML IC ≥ 0.10 AND watchdog has rejected zero promotions in last 60 days	§1A
TASK FUTURE.4	ML target variable change (regression → binary classifier)	After 2026-05-25 decision gate, only if 4 of 4 criteria pass	SOLUTION doc §2.2
TASK FUTURE.5	Trade-outcome dataset construction (~1,250 rows)	After 2026-05-25 gate passes	SOLUTION doc §2.3
TASK FUTURE.6	Stock-specific feature engineering (10 new features)	After 2026-05-25 gate + sample-size ≥ 50 days	SOLUTION doc §2.4
TASK FUTURE.7	Regime-stratified models (BULL/BEAR/SIDEWAYS)	After IC ≥ 0.05 on baseline + bear-regime data observed	SOLUTION doc §2.6
TASK FUTURE.8	Live trading at full capital (₹10L+)	All 4 gate criteria + 30 days at ₹50k scale + zero watchdog alerts	VALIDATION doc §10

Permanently out-of-scope (revisit only with explicit business-case)

- ❌ New engine variants (v5_8, v5_9, etc.) — multi-engine selection inflates Sharpe artificially
- ❌ Strategy diversification (options, futures, F&O) — single-asset focus until equity strategy validated
- ❌ Cross-asset extension (commodities, currencies) — same reason

What to do if the user asks

If during the observation window (2026-04-29 → 2026-05-25) the user requests ANY of TASK FUTURE.1-8: 1. Point them at this section and the unlock condition 2. Show them the current ML IC from `python3 scripts/ml-training-watchdog.py --status` 3. If unlock condition not met, refuse the task with: "This is gated to . Currently . Please consult the IMPLEMENTATION_BRIEF §5 unlock table." 4. Do NOT implement on a "let's just try it" basis — that's the trap §1A warns against

---

6. Test Plan

Unit tests (add to existing test suite)

1. `test_short_block_active_bullish_gap_up` — should return True when bias=BULLISH, gap_up=0.75%, minutes=30
2. `test_short_block_inactive_at_70_minutes` — should return False after the 60-min window
3. `test_rearm_consumes_slot` — first re-arm succeeds, fourth fails
4. `test_rearm_only_on_target` — STOPLOSS exit doesn't create re-arm slot
5. `test_flat_force_exit_at_1330` — flat position exits at 13:30 with reason FLAT_FORCE_EXIT
6. `test_cost_for_trade_typical` — 50 qty × ₹100 entry × ₹100 exit at 12 bps → ₹6 cost
7. `test_weekly_tracker_handles_empty_dir` — runs without crash on engine with 0 reports

Integration test (manual, Tuesday 2026-04-28 EOD)

1. Run all engines through Tuesday's market (auto)
2. Verify Tuesday's report contains: - `[SHORT_BLOCK]` log entry IF Tuesday opens with bullish gap up - `[RE-ARM]` log entry IF any stock TARGETs and re-signals - `FLAT_FORCE_EXIT` reason in the exit-type breakdown IF any flat positions held to 13:30 - Both gross and net P&L in the report
3. Compare Tuesday net P&L to what it would have been without changes (manual estimate based on Track A projections)

DevPilot test pipeline integration

Per `~/.claude/rules/dp-test-pipeline.md` : - Register a new test suite in `project_test_config` for these unit tests - Trigger pipeline on `task_done` for the Track A commit - Verify test artifacts land in `docs/test-runs/results/2026-04-28/pipeline-*/`

---

7. Risk & Rollback Plan

Per-task rollback

Each Phase 1 task is a separate commit. If any task degrades performance: 1. `git revert <task-commit>` 2. Restart engines 3. Document in `docs/observation_journal.md` what reverted and why 4. The other Phase 1 tasks remain in place

Phase 1 wholesale rollback (worst case)

If after 5 trading days under new logic, ALL surviving engines show worse mean P&L than the prior 5 days: 1. Revert all 4 Track A commits in reverse order 2. Re-run the weekly tracker — confirm reverting fixes the regression 3. Re-read the validation doc — possibly the framework was wrong

Engine-failure rollback

If a single engine starts crashing: 1. Disable it (`launchctl unload ...`) 2. Other engines continue running 3. Investigate the crash with logs only — DO NOT change code mid-observation-window

---

8. Memory & Reference Updates

Update these files after Phase 1 completes:

- `MEMORY.md`:
- Add: `[Track A shipped 2026-04-28](feedback_track_a_shipped.md)` — Observation phase begins. No engine changes until 2026-05-25.
- `~/.claude/projects/-Users-soumyaswain/memory/feedback_track_a_shipped.md`:
- Type: feedback
- Body: "Track A tactical fixes shipped 2026-04-28. **Why:** validation doc shows promising-but-unconfirmed signal. Premature ML rebuild = curve-fit on noise. **How to apply:** during 2026-04-29 → 2026-05-25, refuse engine code changes. Refer user to IMPLEMENTATION_BRIEF_2026-04-27.md §3."
- `learnings/` (DevPilot DB):
- Insert: `dp learn "OBSERVATION_DISCIPLINE: 11 trading days insufficient for ML rebuild. Lopez de Prado deflated Sharpe + 95% CI > 0 are the real validation. Track A ships, then 4-week freeze." --tags tradepilot,validation,ml`

---

9. Open Questions to Confirm with Soumya

Before starting Phase 1 work, confirm with Soumya:

1. **OK to disable v5_3 and v4 schedulers?** They're the dead engines. Reversible.
2. **OK with default cost-model bps = 12?** Conservative for Indian retail. Can make it 10 if you have Zerodha-specific numbers.
3. **OK with the Monday 9 AM tracker schedule?** Or prefer Sunday evening for weekend review?
4. **OK to skip ML work for 4 weeks?** This is the hardest one — you may feel the urge to retrain.
5. **OK with the watchdog's hard "PROMOTE" requirement?** Every model promotion needs you (or whoever you delegate) to type "PROMOTE" at the terminal. No CI auto-merge. No `-y` flag. No env override. **This is the user's directive: "no misinformation or wrong training should go to the ML until we confirm it."** Confirmed.
6. **OK with the auto-disable on negative rolling IC?** When 5-day rolling IC < 0.0, watchdog writes `.ml_disabled` flag → composite scorer reads it → ML sub-score weight goes to 0 (rules-only mode). Recoverable by deleting the flag file once IC recovers. The alternative (no auto-disable) means a degraded model continues feeding bad scores until human notices.

---

10. Success Definition

Phase 1 is successful if: - All 4 Track A code changes ship without breaking existing engines - Two retired engines stop generating logs - Weekly tracker runs every Monday and shows current stats

Phase 2 is successful if: - 30 trading days accumulate under the new logic - At least one drawdown observed AND recovered - No code changes made to engines

Phase 3 is successful if: - Decision gate is honestly applied (no rationalizing past failed criteria) - Action taken matches the matrix (no half-measures)

Overall mission is successful if: - By 2026-08-01 we either have a validated money-making system OR honestly know that we don't, with clean data showing why.

Appendix A – File Manifest (deliverables of Phase 1)

FILE	STATUS	OWNER
`scripts/v5-paper-trade.py`	Modified — 4 features added	Implementer
`scripts/weekly-stats-tracker.py`	New	Implementer
`scripts/ml-training-watchdog.py`	**New (Task 1.7)**	Implementer
`scripts/ml-approve.sh`	**New (Task 1.7)**	Implementer
`scripts/retrain-ml.sh`	Modified — wired to watchdog	Implementer
`prototype/v4/models/staging/`	**New directory (Task 1.7)**	Implementer
`~/Library/LaunchAgents/com.tradepilot.weekly-tracker.plist`	New	Implementer
`~/Library/LaunchAgents/com.tradepilot.ml-drift.plist`	**New (Task 1.7)**	Implementer
`~/Library/Logs/tradepilot-ml-drift.log`	Auto-created by drift check	System
`logs/ml-drift-rolling.json`	Auto-created — rolling 30-day IC log	System
`docs/RETIRED_ENGINES.md`	New	Implementer
`docs/observation_journal.md`	New (empty template, fill weekly)	Soumya
`tests/test_track_a.py`	New	Implementer
`tests/test_ml_watchdog.py`	**New (Task 1.7) — 4 acceptance tests**	Implementer

Appendix B – Existing Reference Docs (read for context, do NOT modify)

- `docs/2026-04-27_DEEP_DIVE_60K.md` + `.pdf`
- `docs/2026-04-27_SOLUTION_AND_ML_PLAN.md` + `.pdf`
- `docs/2026-04-27_VALIDATION_HONEST.md` + `.pdf`
- `docs/SHORT_ARM_DIAGNOSIS.md`
- `docs/SHORT_ARM_RESEARCH_BRIEF.md`

End of brief. When you (the implementing Claude) complete each task, update the checkboxes in §2 and the sign-off checklist. Commit changes per task. Run the weekly tracker after Phase 1 to establish the baseline.

If you encounter ambiguity, the validation doc (`2026-04-27_VALIDATION_HONEST.md`) is the source of truth on what the data supports vs aspires to.

If you find yourself wanting to do something not explicitly in this brief — STOP. The four-week observation window only works if no one (including you) modifies the engines during it. Bring questions to Soumya, don't act.

Good hunting.